

Security Audit

Rebase Finance (Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	14
Conclusion	15
Our Methodology	16
Disclaimers	18
About Hashlock	19

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Rebase Finance team partnered with Hashlock to conduct a security audit of their Rebase.sol and ReToken.sol smart contracts. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

Rebase Finance is a decentralized finance (DeFi) platform focusing on elastic supply tokens. These tokens adjust their supply algorithmically based on market conditions, aiming to maintain a stable value. The platform offers various tools and services for users to participate in the DeFi ecosystem, including staking, yield farming, and liquidity provision. Rebase Finance aims to provide innovative token stability and value preservation solutions in the volatile cryptocurrency market.

Project Name: Rebase Finance

Compiler Version: ^0.8.20

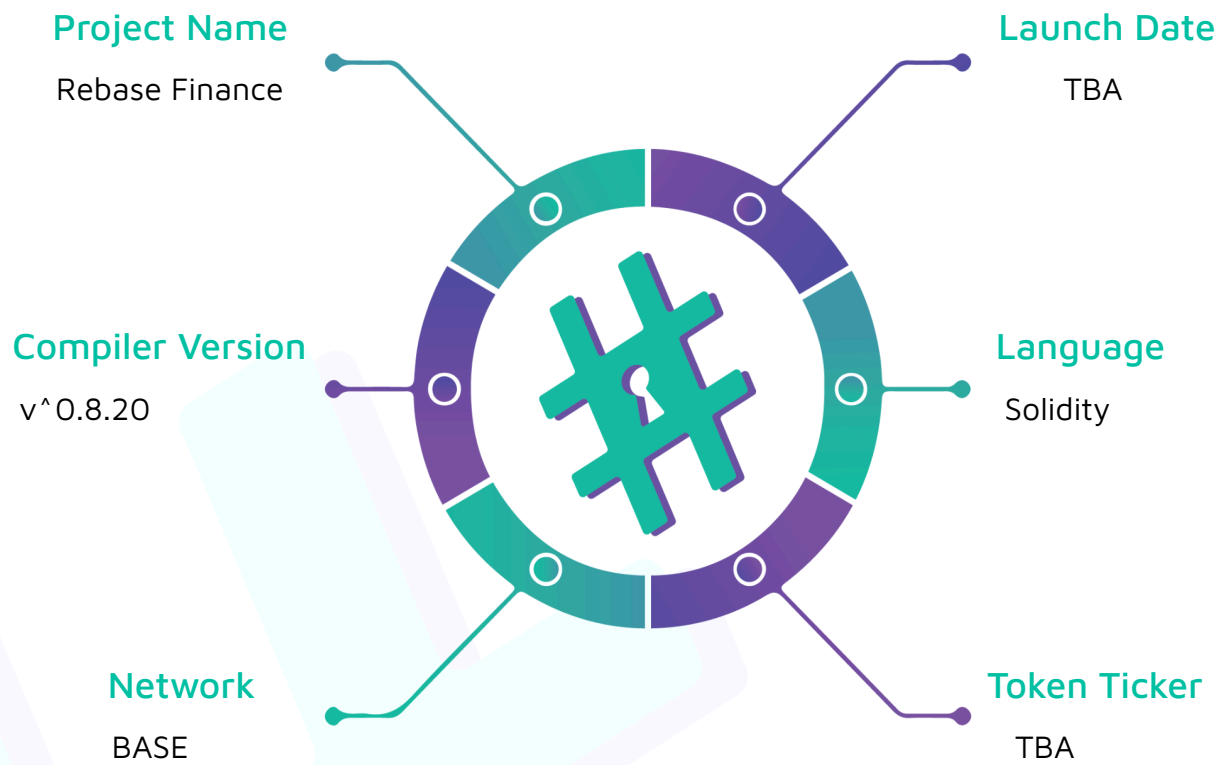
Website: www.rebase.finance

Logo:

The logo for Rebase Finance, featuring the word "Rebase" in a stylized, italicized serif font. The background of the page features a large, faint, light blue gear-like shape.

 **Hashlock.**

Hashlock Pty Ltd

Visualised Context:

Project Visuals:

Rebase

\$1,747,948 is currently staked in Rebase: a protocol for launching tokens to holders of other tokens. [Learn more](#)

Select...

\$REFI is live! Stake any of the tokens above to earn \$REFI. Unstake at any time.

My stakes

About

Rebase is a new protocol for launching tokens. You start by selecting which token communities, e.g. \$BUILD, to launch your token to. Holders of those tokens can then stake to earn your token.

This site is a frontend to the [Rebase contract](#). The contract lets you stake funds once, then restake those funds across different tokens.

There are no fees for staking / restaking / unstaking using Rebase. Staked funds can be unstaked at any time.

\$REFI

[\\$REFI](#) is the protocol token, launched using the Rebase protocol. It has a supply of 2,000,000,000 (2B). 75% is distributed over five weeks to anyone who stakes the following:

May 30 thru July 4:

- 10% - \$AERO
- 10% - \$BUILD
- 10% - \$DEGEN
- 10% - \$ETH

Audit scope

We at Hashlock audited the solidity code within Rebase Finance, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Rebase Finance Smart Contracts
Platform	Base / Solidity
Audit Date	June, 2024
Contract 1	Rebase.sol
Contract 1 MD5 Hash	82efb29735090192eb55fc3fa9b5080d
Contract 2	ReToken.sol
Contract 2 MD5 Hash	def68ad2590ad7d88201712878504a4d
GitHub Commit Hash	11eed8c8dae8e2ba2e7891dc0f8e20f3fcdc09b0

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We have identified some significant vulnerabilities that have been addressed prior to launch.

Hashlock found:

1 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
Rebase.sol - Allows users to: - Stake/Unstake tokens to apps - Restake/Unrestake tokens to apps	Contract achieves this functionality.
ReToken.sol - Allows deployer to - Mint/Burn tokens	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of Rebase Finance, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Rebase Finance smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help us understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

Medium

[M-01] Rebase#unrestake - Lack of checking the return value from the `_unrestake` function

Description

The `unrestake` function basically calls internal function `_unrestake` but the `_unrestake` function doesn't return any value and the `unrestake` function doesn't check the return result so execution of the `unrestake` function will succeed even if calling to the app for unrestaking is failed.

Vulnerability Details

The `_unrestake` function basically makes a low-level call to the app contract:

```
function _unrestake(address app, address token, uint quantity) internal {
    (bool success,)= app.call(
        abi.encodePacked(
            Rebased.unrestake.selector,
            abi.encode(msg.sender, token, quantity)
        )
    );
    emit Unrestake(app, msg.sender, token, quantity, success);
}
```

If calling to the app contract fails, the `success` will be false. But this `success` is not returned so the whole transaction will succeed.

Impact

This issue can cause unexpected results on the entire system.

Recommendation

It's recommended to add a return value into the `_unrestake` function and check the return value in every function calling the `_unrestake` function same as the `_restake`.

Status

Acknowledged

Low

[L-01] Rebase#stake, unstake - Return values of `transfer` / `transferFrom` functions not checked

Description

The `stake` and `unstake` functions call the `transfer` and `transferFrom` functions of `token` contract.

Not all ERC20 implementations perform reverting when there is an error in `transfer` or `transferFrom`.

If the return value is not checked, a transaction that should be marked as failed may succeed without an actual transfer.

Recommendation

Check the return values of the `transfer` and `transferFrom` functions or use the `safeTransfer` and `safeTransferFrom` functions instead.

Status

Resolved

Centralisation

Rebase Finance prioritizes security and utility, relying on the internal team for certain functions. While decentralized, it values trustlessness over correctness, and its restaking functionality may lead to issues with third-party apps not adhering to guidelines.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, Rebase Finance seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd